# Docker Compose 寫法

## 大綱


1. 基本概念
2. 檔案結構
3. 版本（Version）
4. Services 設定
5. Volumes 設定
6. Networks 設定
7. 環境變數（Environment）
8. 常用指令
9. 進階用法
10. 實際範例


---

## 1. 基本概念

### 1.1 什麼是 Docker Compose?

Docker Compose 是一個工具，用於定義和執行多容器 Docker 應用程式。透過 YAML 檔案（預設為 `docker-compose.yml`），可以一次性啟動、停止和管理多個服務。

### 1.2 為什麼要使用 Docker Compose?

- **簡化多容器管理**：一次指令啟動所有相關服務
- **環境一致性**：開發、測試、生產環境統一管理
- **方便重現**：容易分享與版本控制
- **網路自動化**：自動建立容器間的網路

### 1.3 Docker Compose V1 vs V2

- **V1**：`docker-compose`（Python 編寫）
- **V2**：`docker compose`（Go 編寫，已整合至 Docker CLI）
- **建議**：使用 V2（`docker compose`）

---

## 2. 檔案結構

### 2.1 預設檔案名稱

```
docker-compose.yml           # 預設主檔案
docker-compose.yaml          # 也支援 .yaml 副檔名
docker-compose.override.yml # 自動覆蓋主檔案的設定（開發環境用）
docker-compose.prod.yml      # 生產環境設定
```

### 2.2 基本檔案結構

```
version: "3.8"  # 指定 Compose 檔案格式版本

services:        # 定義所有服務（容器）
  web:
    image: nginx:latest
    ports:
      - "80:80"
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: secret

volumes:         # 定義資料卷
  db-data:

networks:        # 定義網路
  backend:
```

---

## 3. 版本（Version）

### 3.1 版本對照表

| 版本 | Docker Engine | 特色 |
|---|---|---|
| 3.8 | 19.03.0+ | 支援 GPU、更好的效能 |
| 3.7 | 18.06.0+ | 新增 healthcheck |
| 3.3 | 17.06.0+ | 支援 secrets |
| 2.4 | 17.12.0+ | 相容舊版 |

### 3.2 版本選擇建議

- **建議使用 3.8**：現代化專案首選
- **特殊需求**：根據 Docker Engine 版本選擇
- **舊專案**：可使用 2.4 或 3.3 確保相容性

---

## 4. Services 設定

### 4.1 基本服務設定

```
services:
  web:
    image: nginx:latest              # 指定 Docker Image
```

```
    container_name: my-web          # 自訂容器名稱
    restart: unless-stopped         # 重啟策略
```

### 4.2 重啟策略（restart）

| 策略 | 說明 |
|---|---|
| `no` | 不自動重啟（預設） |
| `always` | 永遠重啟 |
| `on-failure` | 失敗時重啟，可指定次數 |
| `unless-stopped` | 除非手動停止，否則重啟 |

### 4.3 映像檔來源

```
services:
  web:
    # 使用官方映像檔
    image: nginx:1.25

  app:
    # 從 Dockerfile 構建
    build:
      context: ./app
      dockerfile: Dockerfile
    image: my-app:latest  # 指定建構後的映像檔名稱
```

### 4.4 連接埠映射（ports）

```
services:
  web:
    ports:
      - "80:80"           # 格式: 主機:容器
      - "443:443"
      - "8080:80"         # 可以不同
```

### 4.5 目錄掛載（volumes）

```
services:
  web:
    volumes:
      - ./html:/usr/share/nginx/html:ro  # 主機目錄:容器目錄
      - db-data:/var/lib/mysql           # 命名資料卷
```

### 4.6 環境變數（environment）

```yaml
services:
  db:
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: app
      MYSQL_USER: user
      MYSQL_PASSWORD: password
    # 或使用陣列格式
    # - MYSQL_ROOT_PASSWORD=secret
```

### 4.7 依賴關係（depends_on）

```yaml
services:
  web:
    depends_on:
      - db
      - redis

  db:
    image: mysql:8.0

  redis:
    image: redis:alpine
```

### 4.8 完整服務設定範例

```yaml
services:
  web:
    image: nginx:latest
    container_name: web-server
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    environment:
      - NGINX_HOST=localhost
      - NGINX_PORT=80
    depends_on:
      - app
    networks:
      - frontend

  app:
```

```yaml
    build:
      context: ./app
      dockerfile: Dockerfile
    container_name: app-server
    restart: unless-stopped
    environment:
      - DB_HOST=db
      - DB_PORT=3306
    volumes:
      - app-data:/app/data
    networks:
      - frontend
      - backend

  db:
    image: mysql:8.0
    container_name: mysql-db
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: app
    volumes:
      - db-data:/var/lib/mysql
    networks:
      - backend

volumes:
  app-data:
  db-data:

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
```

---

# 5. Volumes 設定

## 5.1 掛載類型

| 類型 | 格式 | 說明 |
|---|---|---|
| 綁定掛載（bind） | `./host:/container` | 主機特定目錄 |
| 命名資料卷（named volume） | `my-volume:/container` | Docker 管理 |
| tmpfs | `tmpfs:/container` | 記憶體掛載 |

## 5.2 命名字卷定義

```yaml
volumes:
  db-data:            # 命名資料卷
  app-data:
  logs:

services:
  db:
    volumes:
      - db-data:/var/lib/mysql
```

### 5.3 掛載選項

```yaml
services:
  web:
    volumes:
      # 唯讀掛載
      - ./html:/usr/share/nginx/html:ro
      # 讀寫掛載（預設）
      - ./data:/app/data
      # 使用命名資料卷
      - db-data:/var/lib/mysql
```

---

## 6. Networks 設定

### 6.1 網路驅動程式

| 驅動程式 | 說明 |
|---|---|
| `bridge` | 預設，適用於獨立容器 |
| `host` | 移除網路隔離，使用主機網路 |
| `overlay` | 跨 Docker 主機通訊（Swarm 模式） |
| `none` | 禁用所有網路 |

### 6.2 自訂網路

```yaml
networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
  external-net:
    external: true  # 使用外部已存在的網路
```

### 6.3 網路範例

```yaml
services:
  web:
    networks:
      - frontend
      - backend

  app:
    networks:
      - backend

  db:
    networks:
      - backend

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
```

---

## 7. 環境變數（Environment）

### 7.1 直接指定

```yaml
services:
  db:
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: app
      MYSQL_USER: user
      MYSQL_PASSWORD: password
```

### 7.2 使用 .env 檔案

建立 `.env` 檔案：

```
MYSQL_ROOT_PASSWORD=secret
MYSQL_DATABASE=app
MYSQL_USER=user
MYSQL_PASSWORD=password
```

Compose 檔案：

```yaml
services:
  db:
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
```

### 7.3 環境檔案（env_file）

```yaml
services:
  web:
    env_file:
      - ./web.env
      - ./common.env
```

---

## 8. 常用指令

### 8.1 基本指令

```bash
# 啟動所有服務（前景）
docker compose up

# 啟動所有服務（後台）
docker compose up -d

# 啟動特定服務
docker compose up -d web

# 停止並移除服務
docker compose down

# 停止服務（保留資料卷）
docker compose stop

# 檢視服務狀態
docker compose ps

# 檢視服務日誌
docker compose logs -f web

# 重新建構服務
docker compose up -d --build

# 進入服務終端機
docker compose exec web sh
```

### 8.2 管理指令

```
# 檢視正在執行的容器
docker compose ps

# 檢視所有容器（包括已停止）
docker compose ps -a

# 暫停服務
docker compose pause

# 恢復服務
docker compose unpause

# 重啟服務
docker compose restart web

# 移除已停止的容器
docker compose rm

# 查看即時日誌
docker compose logs -f
```

### 8.3 擴展指令

```
# 擴展服務數量
docker compose up -d --scale web=3

# 注意：若有多個容器映射相同連接埠，需使用 replicas 並配合 HAProxy
```

---

## 9. 進階用法

### 9.1 多環境設定

使用 `docker-compose.override.yml` 覆蓋：

```
# docker-compose.yml（基礎設定）
version: "3.8"
services:
  web:
    image: nginx
    ports:
      - "80:80"
```

```yaml
# docker-compose.override.yml（開發環境覆蓋）
version: "3.8"
services:
  web:
    volumes:
      - ./html:/usr/share/nginx/html
    environment:
      - DEBUG=1
```

```
# 自動合併：docker compose up
# 只使用主檔案：docker compose -f docker-compose.yml up
```

### 9.2 指定環境檔案

```
docker compose --env-file .env.prod up -d
```

### 9.3 Profiles（輪廓）

```yaml
services:
  web:
    image: nginx

  db:
    image: mysql:8.0

  mongo:
    image: mongo:6.0
    profiles:
      - database
```

```
# 只啟動預設服務（不含 mongo）
docker compose up -d

# 啟動包含 database profile 的服務
docker compose --profile database up -d
```

### 9.4 健康檢查（healthcheck）

```yaml
services:
  db:
    image: mysql:8.0
```

```
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 60s
```

### 9.5 建構args（build args）

```
services:
  app:
    build:
      context: ./app
      args:
        - BUILD_MODE=production
        - API_KEY=${API_KEY}
```

---

## 10. 實際範例

### 10.1 LNMP 網站架構

```
version: "3.8"

services:
  nginx:
    image: nginx:1.25-alpine
    container_name: nginx-web
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./html:/usr/share/nginx/html:ro
      - ./logs:/var/log/nginx
    depends_on:
      - php
    networks:
      - app-net

  php:
    build:
      context: ./php
      dockerfile: Dockerfile
    container_name: php-fpm
    restart: unless-stopped
    volumes:
      - ./html:/var/www/html
```

```
    depends_on:
      - db
    networks:
      - app-net

  db:
    image: mysql:8.0
    container_name: mysql-server
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db-data:/var/lib/mysql
    networks:
      - app-net

  redis:
    image: redis:7-alpine
    container_name: redis-cache
    restart: unless-stopped
    volumes:
      - redis-data:/data
    networks:
      - app-net

volumes:
  db-data:
  redis-data:

networks:
  app-net:
    driver: bridge
```

### 10.2 Node.js 應用程式

```
version: "3.8"

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: node-app
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
```

```yaml
      - DB_HOST=db
      - DB_PORT=5432
      - REDIS_HOST=redis
    volumes:
      - ./src:/app/src
    depends_on:
      - db
      - redis
    networks:
      - app-net
    command: ["node", "src/index.js"]

  db:
    image: postgres:15-alpine
    container_name: postgres-db
    restart: unless-stopped
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - pg-data:/var/lib/postgresql/data
    networks:
      - app-net

  redis:
    image: redis:7-alpine
    container_name: redis-cache
    restart: unless-stopped
    volumes:
      - redis-data:/data
    networks:
      - app-net

volumes:
  pg-data:
  redis-data:

networks:
  app-net:
    driver: bridge
```

---

## 總結

- **版本選擇**：建議使用 3.8
- **服務定義**：在 `services` 下定義所有容器
- **網路隔離**：使用自訂網路實現服務隔離
- **資料持久化**：使用命名資料卷
- **環境變數**：使用 `.env` 檔案管理敏感資訊
- **常用指令**：`up`、`down`、`logs`、`exec`